知微工作空间 · 组会汇报

让助手学会「记事」

学习档案写入能力升级

从「只会读」到「能写，但由你授权」

2026 年 10 月 · 约 10 分钟

结论先说

这套系统以前只会「回答」，现在开始「积累」

180

条对话消息

近两个月的真实使用记录

1

条长期记忆

沉淀率几乎为零

3

个新增写入工具

本次升级的核心

问题不在于「答得不好」，而在于**答完就散了**：每次对话都从零开始，读过的、想通的、决定过的，都没有留下来。

为什么要改

聊天机器人 vs. 个人工作空间

现在的形态：问答机器

- 每次对话从零开始
- 用户自己决定记什么
- 用完即走，没有沉淀
- 价值 = 单次回答质量

目标形态：研究资产库

- 用得越久越懂你手上的东西
- 助手参与维护档案状态
- 每次使用留下可复用资产
- 价值 = 时间上的复利

缺的那一环很小，但很关键：助手只能读记忆、只能读任务，不能写。 所以档案永远是用户一个人在填。

核心设计

权限门做在「工具注册层」，不在「提示词层」

做法	机制	可靠性
提示词层	在系统提示词里写「请不要自动写入」	软约束 模型可能不遵守，也无法验证
工具注册层	关闭时三个写入工具**根本不进入 tool schema**	硬约束 模型看不到，就没有任何调用途径

我用了拦截请求的方式验证：关闭时，发给模型的 schema 里**确实不含**这三个工具； 打开后才出现。这个属性有自动化测试兜底。

实现

三个写入工具

工具	作用	关键约束
`remember`	写入一条长期记忆	内容相同不重复写；上限 1200 字符；来源标记为 `agent`
`list_learning_tasks`	列出计划与任务编号、完成状态	只读。勾选前必须先调它取 `plan_id` / `task_id`
`set_task_status`	把任务标记为完成 / 未完成	两个编号必须匹配，否则返回失败并提示先查列表

注意后两个是**刻意的两跳结构**：模型必须先查再改。 它把「模型会不会瞎猜编号」变成了一个可观测、可统计的失败模式。

代码 • tools/archive.py

remember：写入前先去重

```
# 完全相同的内容不重复写入，避免助手把同一句话反复记住
for row in list_memories(100):
    if str(row[1]).strip() == content:
        return ToolResult(
            True,
            "这条内容已经在长期记忆里（编号 %d），本次没有重复写入。" % row[0],
            {"memory_id": row[0], "duplicated": True},
        )

memory_id = add_memory(content, category, source="agent")
```

写入统一走 database 层已有函数，工具内部不直接写 SQL。

代码 · agent.py

权限门：四行代码决定模型能看见什么

```python
# 写入类工具默认不注册：只有用户在这一轮显式开启才加上。
# 这样「对话不会自动写入长期记忆」这条安全约定仍然成立。
if allow_writes:
    tool_names = list(tool_names) + [
        name for name in ("remember", "list_learning_tasks", "set_task_status")
        if name not in tool_names
    ]

schemas = all_schemas(tool_names)      # → 发给模型的 tool schema
```

前端勾选「允许助手记录」→ POST /chat 带 allow_writes: true → 这三个工具才注册。

可追溯

给写权限，就得给「审计」手段

```
# 兼容旧库的增量迁移
_ensure_column(cursor, "memories",
               "source", "TEXT DEFAULT 'user'")

def _ensure_column(cursor, table, column, ddl):
    existing = {row[1] for row in
        cursor.execute("PRAGMA table_info(%s)" % table)}
    if column not in existing:
        cursor.execute("ALTER TABLE %s ADD COLUMN %s %s"
                       % (table, column, ddl))
```

先查再改，幂等；重启自动执行，不需要手工改库。

用户能看到的

- 学习档案里，助手写的那条带 助手写入 标签
- 对话的「过程展示」里能看到每一步工具调用与参数
- 随时可以单条删除，不影响用户自己写的
- 原有 1 条记忆完好，source 自动补为 user

实测

用真实模型跑过的四个场景

输入	模型行为	轮数	说明
「请帮我记住：我在研究多轮工具调用」	调用 `remember`	2	改写成了陈述句后落库
「你好，今天心情不错」	**未调用任何工具**	1	没有过度调用
「把第一项未完成任务标记为完成」	`list_learning_tasks` → `set_task_status`	3	两跳正确，没瞎猜编号
同样要求记住，但**开关关闭**	未调用任何工具	1	工具未注册，写不进去

另外写了 61 项自动检查：工具逻辑 19 项、权限门 16 项、HTTP 层 10 项、端到端 16 项，全部通过。

现场演示

三步就能看到效果

1

点开「智能对话」

输入框左下角勾上**「允许助手记录」**，文字变蓝即生效。状态会记住，刷新不丢。

2

对它说一句「记住」

请帮我记住：我正在研究多轮工具调用，分类算「研究方向」。

回复上方会出现 remember 的执行记录。

3

打开「学习档案」验证

新记忆带橙色**「助手写入」**标签，与你自己写的区分开，可单独删除。

想演示勾选任务，就说「把学习计划里第一项没完成的任务标记为完成」——它会先列清单再改，过程展示里能看到两跳。

边界

这套方案目前的局限

问题	具体表现
写入时机靠模型判断	「什么时候该写」仍由模型理解用户意图决定，提示词只能约束不能保证；极端情况下可能漏写或多写
记忆检索不分相关性	上下文用的是最近 20 条记忆，不是最相关的 20 条；记忆变多后早期重要信息会被挤掉
没有用户隔离	`memories` 表没有 `user_id`，单人使用没问题，多用户会互相看到
线上尚未部署	改动目前只在本机验证通过，服务器上还是旧版本，需要上传后重启

下一步

接下来做三件事

A

引用溯源

论文回答带上页码与原文片段，能回查、能给出处。成本最低，用户价值最直接。

B

评测集

30 条覆盖六类场景的小评测集，把「工具选对没有」变成可统计的指标，也是研究工作的基础设施。

C

记忆相关性召回

把记忆从「按时间取最近 20 条」改成「按相关性召回」，一个改动同时提升对话和任务规划。

一句话：**先让系统可靠地留下东西，再测量它留得好不好。**

附录

导师可能会问的问题

Q 模型会不会乱写、污染档案?

A 写权限默认关闭，关闭时工具根本不注册；开启后也要求用户明确提出，且写入来源标记为 agent、可单条删除。三道防线：注册层、提示词、可审计。

Q 为什么不用向量检索记忆?

A 当前记忆量级是几十条，取最近 20 条够用；相关性召回已列入下一步。它的收益在记忆变多以后才显现。

Q 模型调用失败怎么办?

A 工具异常统一返回失败而不是抛出，让模型有机会换策略；同时有轮数、单轮工具数、总耗时、token 四重预算上限。

Q 这和只改 prompt 有什么区别?

A prompt 是软约束，模型可能不遵守；不注册的工具模型看不到，属于硬约束。我用拦截请求验证过 schema 里确实没有这些工具。

Q 怎么证明它「真的有用」?

A 目前是功能级验证（61 项检查 + 四场景实测）。效果级证明需要评测集，这是下一步要补的短板。

Q 这些改动会破坏已有数据吗?

A 不会。迁移只新增一列并有默认值，重启自动执行；实测原有记忆完好，source 自动补为 user。